

2차 과제에서 만든 야구 타자 성적 관리 시스템을 3차 과제 조건인 반복문과 함수 중심 구조로 어떻게 확장하면 좋을까?

좋아요 승우님.

2차 과제에서 만든 야구 타자 성적 관리 시스템은 이미 **입력 → 계산 → 출력 → 등급/강타자 판정** 흐름이 완성되어 있으므로, 3차 과제에서는 이 기능들을 그대로 유지하되 **반복문과 함수 중심 구조**로 바꾸면 됩니다.

가장 먼저 `while(1)` 무한 반복문을 사용해서 메인 메뉴를 만들면 좋아요. 사용자가 프로그램을 실행하면 메뉴가 계속 출력되고, 원하는 번호를 선택해 기능을 실행하도록 구성합니다. 예를 들면 1. 타자 정보 입력, 2. 현재 타율 계산, 3. 타자 성적 출력, 4. 타자 등급 및 강타자 여부 분석, 0. 프로그램 종료 처럼 구성할 수 있습니다.

또한 2차 과제에서는 대부분의 코드가 `main()` 함수 안에 길게 작성되어 있었다면, 3차 과제에서는 기능별로 함수를 분리하는 것이 핵심입니다. 예를 들어 `print_menu()` 는 메뉴 출력, `input_player()` 는 타자 정보 입력, `calculate_stats()` 는 타율 계산, `print_player_info()` 는 성적 출력, `print_grade()` 와 `analyze_power_hitter()` 는 등급 및 강타자 분석을 담당하도록 나눌 수 있습니다.

이렇게 하면 `main()` 함수 안에서는 메뉴 번호를 입력받고, 선택된 번호에 따라 필요한 함수를 호출하는 방식으로 코드가 훨씬 간결해집니다. 즉, `main()` 은 전체 흐름을 관리하고, 실제 기능은 각각의 함수가 담당하는 구조가 됩니다.

3차 과제 필수 조건에 맞추려면 최소 3개 이상의 함수를 정의하고 호출해야 하며, 함수 중 하나 이상은 매개변수를 받아야 하고, 하나 이상은 `return` 으로 결과를 반환해야 합니다. 예를 들어 `calculate_average(float f_hit, float f_at_bat)` 함수는 안타 수와 타수를 매개변수로 받아 현재 타율을 계산한 뒤 `return` 으로 반환하도록 만들 수 있습니다.

정리하면, 3차 과제에서는 기존 V2.0의 조건문 기능은 유지하면서 **메뉴 시스템 + 함수 분리 + 반복 실행 구조**를 추가하는 방향으로 확장하면 가장 자연스럽습니다.

while(1) 무한 반복문을 사용할 때 프로그램이 정상적으로 종료되게 하려면 break를 어디에 넣어야 할까?

while(1) 무한 반복문을 사용할 때는 사용자가 프로그램 종료를 원하는 순간 반복문을 빠져나올 수 있도록 break 문을 넣어야 합니다. 보통은 메뉴 시스템에서 사용자가 종료 번호를 입력했을 때 break가 실행되도록 구현합니다.

예를 들어 야구 타자 성적 관리 시스템에서는 메뉴를 출력한 뒤 사용자가 번호를 입력하면, if 또는 else if 조건문으로 메뉴를 구분하게 됩니다. 이때 종료 메뉴인 0을 선택했을 경우 아래처럼 break를 실행하도록 구성하면 됩니다.

```
<> C
else if (menu == 0)
{
    printf("\n프로그램을 종료합니다.\n");
    break;
}
```

이 코드는 반드시 while(1) 반복문 내부에 있어야 합니다. 사용자가 0을 입력하면 break가 실행되면서 무한 반복문을 빠져나오고, 이후 main() 함수의 마지막에 있는 return 0;으로 프로그램이 정상 종료됩니다.

만약 break가 없으면 while(1)은 종료 조건이 없는 반복문이기 때문에 프로그램이 계속 메뉴를 출력하며 무한히 반복됩니다. 따라서 3차 과제에서는 메뉴 시스템 안에 종료 메뉴와 break문을 함께 구현하는 것이 중요합니다.



📄 👍 🗨️ ↗️ ↻️ ...

calculate_average() 함수에서 계산한 타율 값을 다른 함수에서도 사용하려면 전역변수와 return을 어떻게 사용하는 게 좋을까?

calculate_average() 함수에서 계산한 현재 타율 값을 다른 함수에서도 사용하려면, 계산 결과를 return 으로 반환하고 그 값을 전역변수에 저장하는 방식이 가장 자연스럽습니다.

예를 들어 calculate_average() 함수는 안타 수와 타수를 매개변수로 받아 현재 타율을 계산한 뒤 return 문으로 결과를 반환하도록 만들 수 있습니다.

```
⟨⟩ C

float calculate_average(float f_hit, float f_at_bat)
{
    return f_hit / f_at_bat;
}
```

그리고 calculate_stats() 함수 안에서 이 함수를 호출하여 반환값을 전역변수인 current_avg 에 저장합니다.

```
⟨⟩ C

current_avg = calculate_average(f_hit, f_at_bat);
```

이렇게 하면 current_avg 가 전역변수이기 때문에 프로그램 전체에서 값을 공유할 수 있습니다. 따라서 print_player_info(), print_grade(), analyze_power_hitter() 같은 다른 함수에서도 현재 타율 값을 다시 계산하지 않고 바로 사용할 수 있습니다.

즉, return 은 함수 내부에서 계산한 결과를 밖으로 전달하는 역할을 하고, 전역변수는 여러 함수가 공통 데이터를 공유할 수 있도록 하는 역할을 합니다. 이번 3차 레에서는 두 가지를 함께 사용하면 함수 구조를 깔끔하게 유지하면서도 데이터 전달을 효율적으로 구현할 수 있습니다.

입력, 계산, 출력, 분석 기능을 각각 함수로 분리하면 main 함수는 어떤 역할만 담당하도록 구성하는 게 좋을까?

입력, 계산, 출력, 분석 기능을 각각 함수로 분리하면 `main()` 함수는 **전체 프로그램의 흐름을 관리하는 역할**만 담당하도록 구성하는 것이 좋습니다.

즉, `main()` 안에 입력 코드나 계산식, 긴 출력문을 모두 작성하는 것이 아니라, 메뉴를 출력하고 사용자의 선택에 따라 필요한 함수를 호출하는 방식으로 구성합니다.

예를 들면 다음과 같습니다.

« C



```
while (1)
{
    print_menu();
    scanf("%d", &menu);

    if (menu == 1)
    {
        input_player();
    }
    else if (menu == 2)
    {
        calculate_stats();
    }
    else if (menu == 3)
    {
        print_player_info();
    }
    else if (menu == 4)
```



```
else if (menu == 4)
{
    print_grade(current_avg);
    analyze_power_hitter(current_avg, home_run);
}
else if (menu == 0)
{
    break;
}
}
```

이렇게 하면 `main()` 함수는 **메뉴 선택과 함수 호출**만 담당하게 되고, 실제 세부 기능은 각각의 함수가 처리하게 됩니다.

이 구조의 장점은 코드가 훨씬 읽기 쉬워지고, 나중에 기능을 수정하거나 추가할 때도 해당 함수만 고치면 된다는 점입니다. 특히 3차 과제처럼 반복문과 함수 활용이 핵심인 경우에는 `main()` 을 간결하게 구성하는 것이 중요합니다.

📄 🌟 🔖 ↗️ ↺ ...

전역변수와 지역변수를 함께 사용할 때 각각 어떤 데이터를 저장하도록 구분하면 프로그램을 더 효율적으로 만들 수 있을까?

전역변수와 지역변수를 함께 사용할 때는, 프로그램 전체에서 여러 함수가 공통으로 사용해야 하는 데이터는 전역변수로 저장하고, 특정 함수 안에서만 잠시 사용하는 데이터는 지역변수로 저장하는 방식이 가장 효율적입니다.

예를 들어 이번 야구 타자 성적 관리 시스템에서는 타자의 이니셜, 경기 수, 타수, 안타 수, 홈런 수, 현재 타율 같은 값들은 여러 함수에서 공통으로 사용됩니다. 입력 함수에서는 값을 저장하고, 계산 함수에서는 계산에 사용하며, 출력 함수와 분석 함수에서도 다시 사용해야 하기 때문에 이런 데이터들은 전역변수로 선언하는 것이 편리합니다.

반면에 함수 내부에서만 잠깐 사용하는 계산용 변수는 지역변수로 사용하는 것이 좋습니다. 예를 들어 `calculate_stats()` 함수 안에서 사용하는 `f_hit`, `f_at_bat` 같은 변수는 현재 타율을 계산할 때만 필요하므로 지역변수로 선언하는 것이 적절합니다.

이렇게 구분하면 프로그램 구조가 더 명확해지고, 불필요하게 메모리를 공유하는 상황도 줄일 수 있습니다. 또한 지역변수는 해당 함수 안에서만 사용할 수 있기 때문에 다른 함수에서 실수로 값을 변경하는 문제를 방지할 수 있다는 장점도 있습니다.

즉, 전역변수는 여러 함수가 함께 사용하는 “공용 데이터”, 지역변수는 특정 함수 안에서만 사용하는 “임시 데이터” 역할로 구분하면 프로그램을 더 효율적으로 구성할 수 있습니다.

📄 🌟 🔖 ↕ ↻ ...